Homework 5

Product API, Infrastructure, Load Testing, and Observability

Zoya Arief

CS6650 - Building Scalable Distributed Systems
October 18, 2026

Project Summary

This project implements the Product portion of the supplied OpenAPI contract in Go with Gin. It stores products in a concurrency-safe in-memory map, packages the service in Docker, deploys it to Amazon ECS through Terraform, load tests the deployed API with Locust, and adds local Prometheus, Grafana, and structured JSON request logs for RED observability.

Required repository deliverables

Assignment part	Evidence in repository
Part II - Product API	src/product-service.go, src/Dockerfile, api.yaml, response-code examples and screenshots
Part III - Infrastructure	terraform/ provider, variables, modules, main configuration, outputs, and deployment instructions
Part IV - Testing	load_test/ Locust files, HTML/CSV reports, comparison results, and stress-test screenshots
Part V - Observability	docker-compose.yml, Prometheus config, Grafana dashboard export, JSON log excerpt, and screenshots

The repository README is the operational handoff: it contains setup commands, all endpoint examples, AWS deployment and cleanup steps, test results, and observability instructions. This report focuses on the required evidence and explanations used during the mock interview.

Part II - Product API

Contract and request flow

Gin routes requests under /v1 to a ProductStore handler. The handler validates the path and JSON body, locks the map, reads or replaces the existing product, and returns the contract-defined response. GET /v1/products/{productId} retrieves a product. POST /v1/products/{productId}/details replaces details only for an existing product; it does not create a new identifier.

Endpoint	Success	Required error responses
GET /v1/products/{productId}	200 with Product JSON	404 product not found; 500 internal error
POST /v1/products/{productId}/details	204 with no body	400 invalid input; 404 product not found; 500 internal error

Schema and validation

Product contains product_id, sku, manufacturer, category_id, weight, and some_other_id. Numeric IDs must be positive. Weight is a *int32 so validation can distinguish a missing field from the valid value zero. The URL productId must equal body product_id. A sync.RWMutex permits concurrent reads and protects writes.



Figure 1. Local GET 200, GET 404, POST 204, and the updated GET response.



Figure 2. POST 400 for missing required weight and POST 404 for an unknown product.

Mock interview explanation

A Go map gives constant-time lookup for this small prototype, while RWMutex prevents concurrent-map races. Without the lock, simultaneous handlers could race or panic. The map remains process-local, so it would lose data on restart and would not be shared across multiple ECS tasks; a persistent database is required for scale.

Response Codes and AWS Deployment Evidence

Complete response-code coverage

The screenshots and README curl commands cover every response code listed for the two Product endpoints: 200, 204, 400, 404, and 500. The 500 response was exercised through a temporary deliberate-panic test hook and Gin recovery middleware; the hook is disabled in the submitted server.



Figure 3. JSON-formatted 500 responses for GET and POST during the controlled recovery test.



Figure 4. The Terraform-deployed ECS service returning GET 200, POST 204, and the updated product.

Deployment result

Terraform built a linux/amd64 Docker image, pushed the latest tag to Amazon ECR, registered an ECS Fargate task definition, and started one public task on port 8080. The captured public endpoint was 44.204.84.54:8080. After evidence collection, terraform destroy removed the managed resources so the submitted project does not leave the ECS task or related infrastructure running.

How each Terraform file connects

- provider.tf configures AWS and Docker providers and ECR authentication.
- variables.tf centralizes region, names, port 8080, task count, and log retention.
- main.tf connects the network, ECR, logging, and ECS modules, then builds and pushes the image.
- modules/ creates the security group/network lookup, ECR repository, CloudWatch log group, ECS cluster, task definition, and service.
- outputs.tf exposes the cluster and service names needed to inspect the deployment.

What changed from the starter

The starter albums server was replaced with product-service.go plus its Go module files and Dockerfile in src/. Terraform was adapted to build that directory, expose port 8080, name the Product API resources, and pass the ECR image to ECS.

Part IV - Locust Test Design and Client Comparison

Test design

The 50-user comparison used one Locust process, a spawn rate of 10 users/second, a 60-second duration, and a 1-2 second wait between requests. A 3:1 GET-to-POST task ratio models a catalog workload in which reads are more common than product-detail changes. Both clients used the same scenario against the deployed API.

Client	Requests	Failures	RPS	Average	Median	p95	Maximum
HttpUser - 50	1,824	0	31.86	32.73 ms	27 ms	40 ms	1,103 ms
FastHttpUser - 50	1,907	0	31.94	29.55 ms	26 ms	47 ms	269 ms

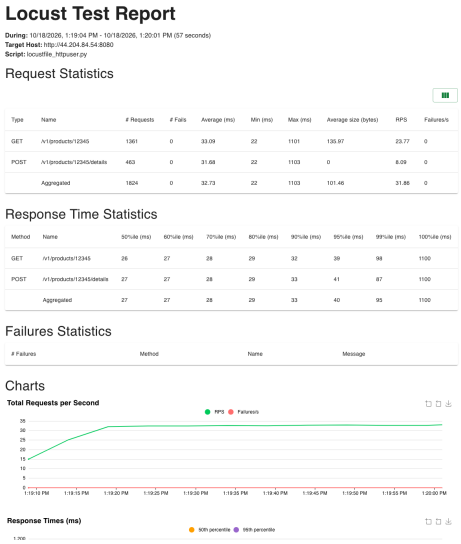


Figure 5. HttpUser at 50 concurrent users.

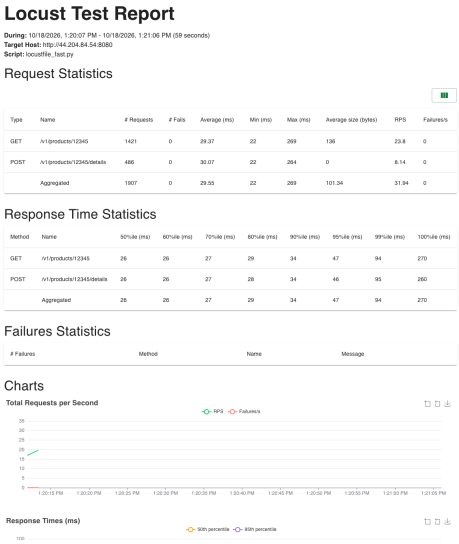


Figure 6. FastHttpUser at 50 concurrent users.

HttpUser versus FastHttpUser

Throughput was nearly identical because the configured 1-2 second user wait, rather than client CPU, limited request generation. FastHttpUser had slightly lower average and median latency and a much lower maximum in this sample. FastHttpUser uses geventhttpclient and has less client-side overhead than the requests-based HttpUser, but Internet and ECS latency also vary between runs.

Part IV - Stress Test and Tradeoffs

Where the server began to struggle

FastHttpUser run	Requests	Failures	RPS	Average	Median	p95	p98	Maximum
200 users	5,661	0	126.54	32 ms	25 ms	41 ms	180 ms	314 ms
500 users	14,020	0	313.35	53.85 ms	26 ms	77 ms	720 ms	2,418 ms

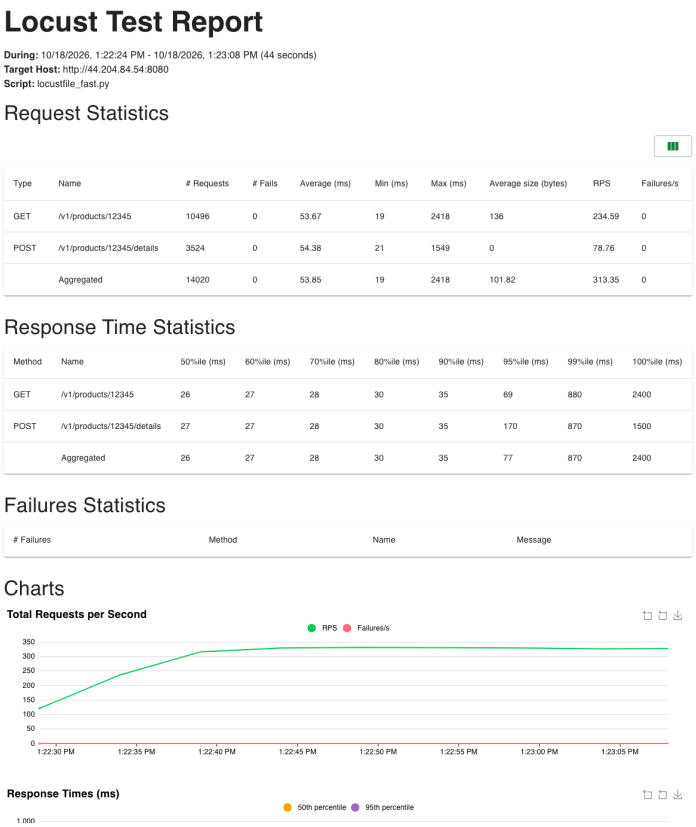


Figure 7. FastHttpUser 500-user stress test. Throughput rose to about 313 RPS with no failures, but tail latency widened sharply.

The service stayed healthy at 200 users. At 500 users, average latency increased to 53.85 ms, p98 reached 720 ms, and the slowest request reached 2.418 seconds. This is the observed struggle point for the single 0.25-vCPU, 512-MiB task: it continued serving requests, but queueing and tail latency became visible. A GET immediately after the run still returned 200.

Tradeoff question: which operations are most common?

Product reads should dominate in a typical store because many customers browse while relatively few catalog updates occur. RWMutex suits this read-heavy prototype because multiple GET handlers can hold read locks together. Each POST requires the exclusive lock and briefly blocks readers and writers. With 25% writes and every request targeting the same key, lock contention can contribute to latency at high concurrency.

What changes for a real system?

The in-memory map is fast and simple but has no durability or cross-task consistency. A real deployment would move products to a database, add a read cache, scale stateless API tasks horizontally, and avoid a process-wide lock becoming the coordination mechanism.

Part V - RED Metrics and Structured Logs

Instrumentation

A Gin middleware records every API request after its handler completes. Labels use the method and Gin route template, preventing a separate high-cardinality metric series for every product ID. The /metrics scrape is excluded so monitoring traffic does not alter application measurements.

RED signal	Metric	Dashboard calculation
Rate	product_api_requests_total	rate(counter[1m]), grouped by method and endpoint
Errors	product_api_errors_total	100 x error rate / request rate; errors are all non-2xx responses
Duration	product_api_request_duration_seconds	histogram_quantile() over buckets for p50 and p95

```
# TYPE product_api_requests_total counter
product_api_requests_total{endpoint="/v1/products",method="GET"} 1
product_api_requests_total{endpoint="/v1/products/",method="GET"} 1
product_api_requests_total{endpoint="/v1/products/:productId",method="GET"} 4
zoyaarief@Zoyas-MacBook-Air ~ %
```

Figure 8. Prometheus exposition showing Product API counters and duration histogram data.

Local stack

docker-compose.yml runs the Product API, Prometheus, and Grafana locally. Prometheus scrapes product-api:8080/metrics, and Grafana reads Prometheus at prometheus:9090. The reusable dashboard export is stored at observability/grafana/product-api-red-dashboard.json.

Request logs

The same middleware emits one JSON object per request containing timestamp, method, endpoint, status, and duration_ms. These fields preserve the same RED evidence at individual-request granularity.

Part V - Idle Versus Loaded Dashboard

The required local observability run used FastHttpUser with 50 users, a 10 users/second spawn rate, the same 3:1 GET-to-POST mix, and a 1-2 second wait. It produced 3,008 requests, zero failures, and 32.73 requests/second. GET processed 2,285 requests and POST processed 723.



Figure 9. Required Grafana evidence: idle, loaded, and return-to-idle behavior for rate, non-2xx percentage, and p50/p95 duration.

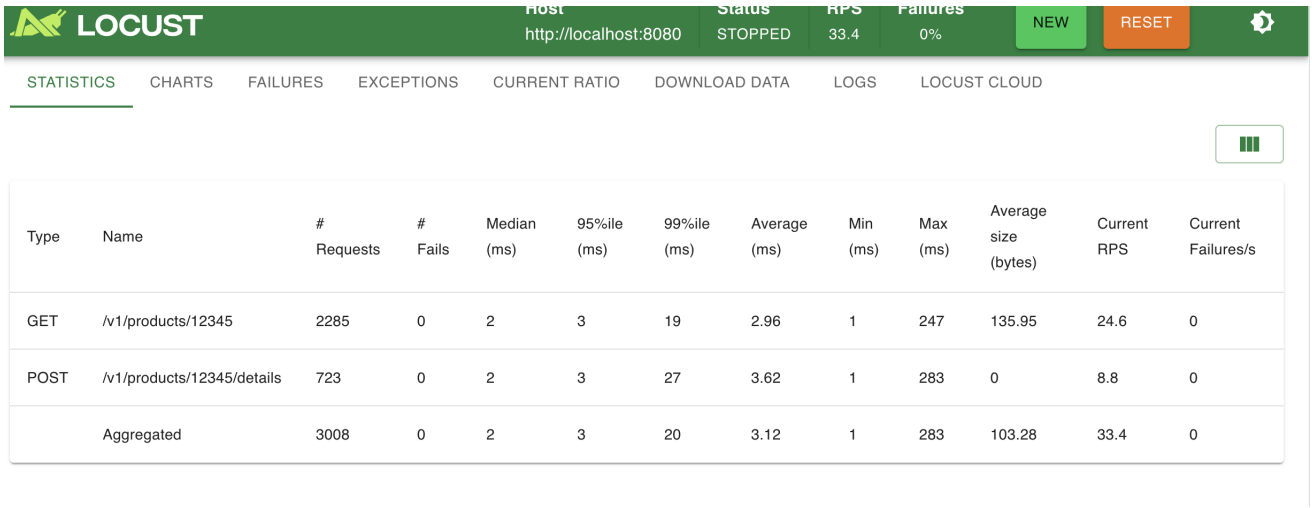
Dashboard interpretation

The request-rate panel rises from zero to approximately 25 GET requests/second and 8 POST requests/second, then returns to zero when Locust stops. The error panel remains at 0%. The duration panel remains stable at about 2.5 ms p50 and 5 ms p95. Locust reported a 2 ms median and 3 ms p95; Grafana estimates quantiles from Prometheus histogram bucket boundaries, so the values need not match exactly.

When to use the dashboard versus logs

Use the Grafana dashboard for live system health, trends, and quickly detecting when traffic, error rate, or latency changes. Use structured logs for after-the-fact debugging and for examining the exact route, status, and duration of individual requests.

Part V - Same-Run Locust and Log Evidence



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/v1/products/12345	2285	0	2	3	19	2.96	1	247	135.95	24.6	0
POST	/v1/products/12345/details	723	0	2	3	27	3.62	1	283	0	8.8	0
	Aggregated	3008	0	2	3	20	3.12	1	283	103.28	33.4	0

Figure 10. Locust statistics for the same run shown in the Grafana dashboard: 3,008 requests and zero failures.

Required request-log excerpt

```
{"duration_ms":0.088792,"endpoint":"/v1/products/:productId/details",
"method":"POST","status":204,"timestamp":"2026-10-18T22:48:56Z"}
{"duration_ms":0.022708,"endpoint":"/v1/products/:productId",
"method":"GET","status":200,"timestamp":"2026-10-18T22:48:56Z"}
{"duration_ms":0.118208,"endpoint":"/v1/products/:productId/details",
"method":"POST","status":204,"timestamp":"2026-10-18T22:48:57Z"}
{"duration_ms":0.032667,"endpoint":"/v1/products/:productId",
"method":"GET","status":200,"timestamp":"2026-10-18T22:48:57Z"}
```

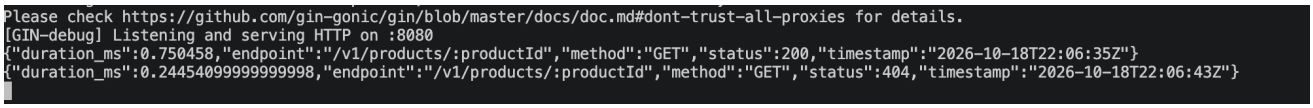


Figure 11. Structured JSON request logs containing endpoint, status, and duration for GET and POST requests.

The full 20-line excerpt from this run is stored in observability/request-log-excerpt.txt. The timestamps overlap the loaded interval shown in Grafana, establishing that the dashboard and logs describe the same run.

Mock Interview Design Questions

How would you design a scalable backend for the complete online-store API?

I would run stateless HTTP services behind a load balancer and scale multiple ECS tasks automatically. The API could begin as a modular service with clear domains - catalog, users, carts, orders, inventory, and payments - and split independently where load or ownership justifies it. Requests would be authenticated at the edge, validated against the API contract, and traced with a request ID.

Transactional orders, inventory, and payments need a managed relational database such as Aurora or RDS, with transactions, backups, and read replicas as needed. Frequently read product data belongs in ElastiCache; images belong in S3 behind a CDN; and product search can use OpenSearch. SQS or EventBridge can decouple order events, inventory updates, notifications, and other background work. Write endpoints should use idempotency keys so client retries do not duplicate an order or payment.

Operationally, secrets belong in Secrets Manager, services use least-privilege IAM, and deployments use health checks and rolling updates. Metrics, structured logs, traces, alarms, and dashboards provide observability. This design replaces the single-process map with durable shared state and permits horizontal scaling and failure recovery.

What does Terraform being declarative mean, and how does it help?

Declarative configuration describes the desired final state - for example, an ECS service with a particular image, task count, port, security group, and log group. Terraform compares that configuration with its state and the provider, builds a dependency graph, and plans only the changes needed to converge on the requested result.

An imperative program instead lists ordered commands such as create a repository, push an image, create a cluster, and start a service. The declarative approach is repeatable, reviewable in version control, easier to reproduce on another machine, and safer to change because terraform plan shows the expected difference before apply. Modules also make the network, ECR, logging, and ECS pieces reusable while dependency references determine creation order.

Final result

The submitted repository contains the required Product API and Dockerfile, reproducible Terraform deployment, examples for every required response code, comparative and stress Locust evidence, and the Part V Grafana and same-run JSON log evidence. The README provides the exact commands needed to reproduce each stage.